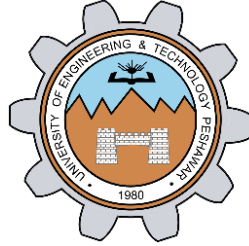


Midterm Paper Codes Analysis



Spring 2024

CSE-308 Digital System Design

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Rehmat Ullah Khattak

Date:

7th April 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Problem 1:

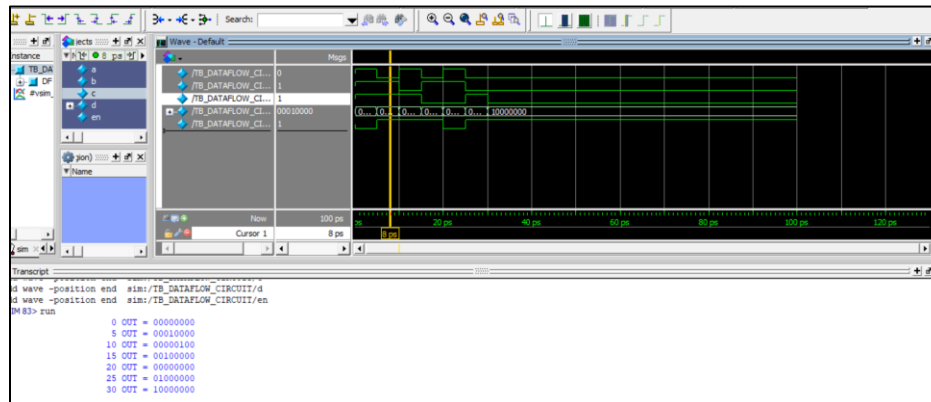
part(a)

DATAFLOW_CIRCUIT

Code:

```
1 module DATAFLOW_CIRCUIT (a, b, c, en, d);
2
3     input a, b, c, en;
4     output [0:7] d;
5
6     wire na, nb, nc;
7
8     assign na = !a;
9     assign nb = !b;
10    assign nc = !c;
11    assign d[0] = na && nb && nc && en;
12    assign d[1] = na && nb && c && en;
13    assign d[2] = na && b && nc && en;
14    assign d[3] = na && b && c && en;
15    assign d[4] = a && nb && nc && en;
16    assign d[5] = a && nb && c && en;
17    assign d[6] = a && b && nc && en;
18    assign d[7] = a && b && c && en;
19
20 endmodule
```

Output:



Analysis:

This code is error-free. The code works as intended and it implements a 3x8 decoder with **enable** input(using Dataflow Modelling).



part (c):

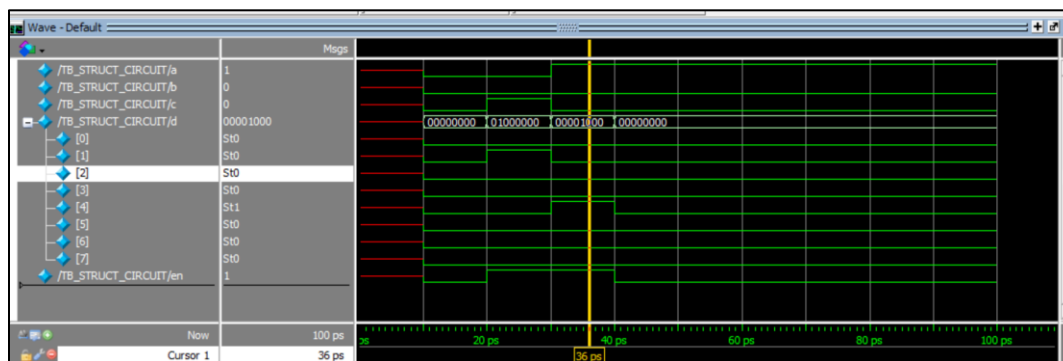
STRUCT_CIRCUIT

Rewrite the DATAFLOW_CIRCUIT using Verilog built-in primitives and structural Verilog. Part of the module is done for you.

Code:

```
1 module STRUCT_CIRCUIT (a, b, c, en, d);
2
3     input a, b, c, en;
4     output [0:7] d;
5     //Write your code here
6
7     wire na, nb, nc;
8
9     not n1 (na, a);
10    not n2 (nb, b);
11    not n3 (nc, c);
12    and a1 (d[0], na, nb, nc, en);
13    and a2 (d[1], na, nb, c, en);
14    and a3 (d[2], na, b, nc, en);
15    and a4 (d[3], na, b, c, en);
16    and a5 (d[4], a, nb, nc, en);
17    and a6 (d[5], a, nb, c, en);
18    and a7 (d[6], a, b, nc, en);
19    and a8 (d[7], a, b, c, en);
20
21 endmodule
```

Output:



Analysis:

This code is fine and error-free. The code works as intended and it implements a 3x8 decoder with **enable** input(using gate-level modelling).



part (d):

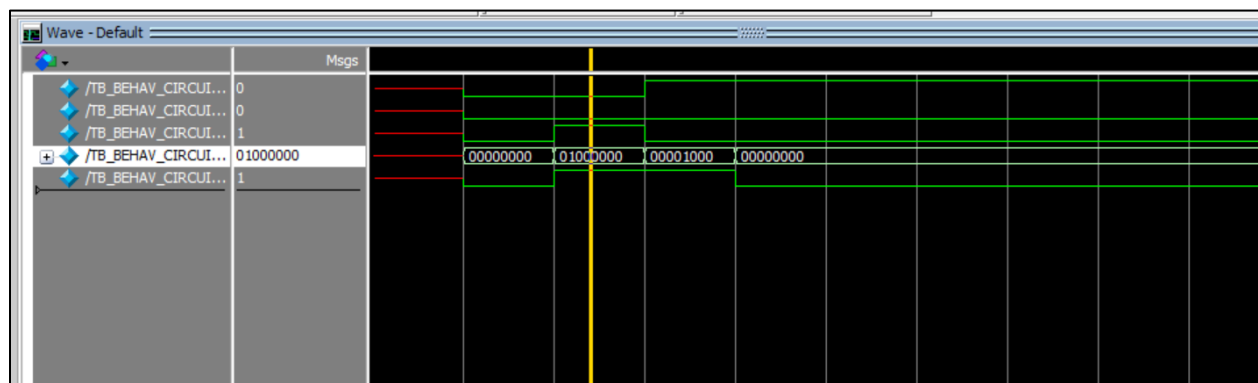
BEHAV_CIRCUIT

Rewrite the DATAFLOW_CIRCUIT using behavioral Verilog. Part of the module is done for you.

Code:

```
1  module BEHAV_CIRCUIT (a, b, c, en, d);
2      input a, b, c, en;
3      output [0:7] d;
4      //Write your code here
5
6      reg [0:7] d;
7      reg na, nb, nc;
8
9      always @(*)
10     begin
11         na = !a;
12         nb = !b;
13         nc = !c;
14         d[0] = na && nb && nc && en;
15         d[1] = na && nb && c && en;
16         d[2] = na && b && nc && en;
17         d[3] = na && b && c && en;
18         d[4] = a && nb && nc && en;
19         d[5] = a && nb && c && en;
20         d[6] = a && b && nc && en;
21         d[7] = a && b && c && en;
22     end
23 endmodule
```

Output:



Analysis:

This behavioral version of the above code is error-free. However, it'd have been better to use **switch statement** in this code. The reason is that in behavioral modelling, we describe the **behavior** of the system, and our desired circuit is synthesized. While attempting paper, I implemented the same functionality using **switch statement**.



Problem 2:

part (a):

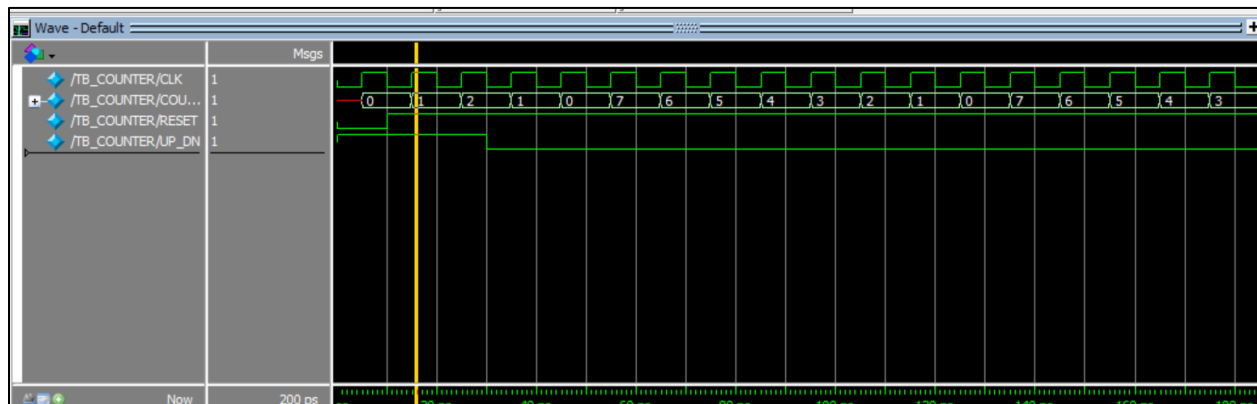
COUNTER

In this module, design a 3-bit up / down counter, a counter that counts in both up and down directions.

Code:

```
1  module COUNTER (CLK, RESET, UP_DN, COUNT);
2      //Write your code here
3
4      input CLK, RESET, UP_DN;
5      output [2:0] COUNT;
6
7      reg [2:0] COUNT;
8
9      always @(posedge CLK)
10         if (~RESET) //Active low RESET
11             COUNT <= 0;
12         else
13             if (UP_DN) //Up mode selected
14                 COUNT <= COUNT + 1; //Increment COUNT
15             else //Down mode selected
16                 COUNT <= COUNT - 1; //Decrement COUNT
17
18     endmodule
```

Output:



Analysis:

This code is error-free and it perfectly models a sequential circuit (3-bit Up/Down Counter). However, it generates a **warning** in xilinx ise because of one **ambiguity** in this code. We are incrementing the counter without considering the overflow case. For example, if we have COUNT = 111 and then we increment it. We would be getting a 4-bit result with a carry.

Line 14: Result of 4-bit expression is truncated to fit in 3-bit target.
Line 17: Result of 32-bit expression is truncated to fit in 3-bit target.

Solution:

Write it in this format.

```
if (UP_DN) //Up mode selected
    //COUNT <= COUNT + 1; //Increment COUNT
    COUNT <= {COUNT + 3'b001}; //Increment COUNT
else //Down mode selected
    //COUNT <= COUNT - 1; //Decrement COUNT
    COUNT <= {COUNT - 3'b001}; //Decrement COUNT
```



part (b):

MUX81

In this module, design a byte-wide 8-to-1 multiplexer. In this case, the value on the 3-bit select line will route 1 of 8 inputs to the output. This module is purely combinatorial.

Code:

```
MUX81.v PISO.v STRUCT_CIRCUIT.v TB_DATAFLOW_CIRCUIT.v TB_PISO.v TB_TOP.v TOP.v TB_S
1  module MUX81 (SEL, D0, D1, D2, D3, D4, D5, D6, D7, OUT);
2      //Write your code here
3
4      input [2:0] SEL;
5      input [7:0] D0, D1, D2, D3, D4, D5, D6, D7;
6      output [7:0] OUT;
7
8      reg [7:0] OUT;
9
10     always @(*)
11     case (SEL)
12         3'b000: OUT = D0;
13         3'b001: OUT = D1;
14         3'b010: OUT = D2;
15         3'b011: OUT = D3;
16         3'b100: OUT = D4;
17         3'b101: OUT = D5;
18         3'b110: OUT = D6;
19         3'b111: OUT = D7;
20     endcase
21
22 endmodule
```

Output:

Its output can be seen in the upcoming last part of this problem. So, no need to attach it here.

Analysis:

This code is error-free, and it perfectly models a combinational circuit (byte-wide 8-to-1 multiplexer).



part (c):

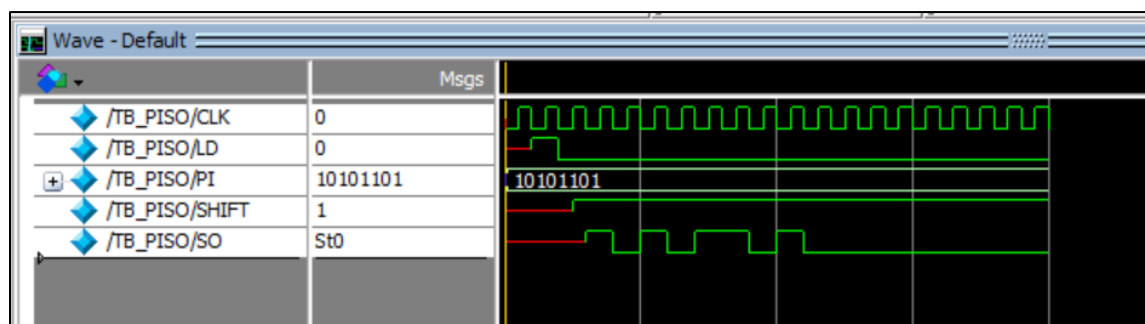
PISO

In this module, create a register that loads data in parallel but shifts data out serially, MSB first.

Code:

```
1  module PISO (CLK, PI, SO, LD, SHIFT);
2      //Write your code here
3
4      input CLK, LD, SHIFT;
5      input [7:0] PI;
6      output SO;
7
8      reg SO;
9      reg [7:0] SHIFTER;
10
11     always @(negedge CLK)
12     begin
13         if (LD)
14             SHIFTER = PI;
15         else if (SHIFT)
16         begin
17             SO = SHIFTER[7];
18             SHIFTER = {SHIFTER[6:0], 1'b0};
19         end
20     end
21 endmodule
```

Output:



Analysis:

This code is error-free, and it perfectly models an 8-bit Parallel In Serial Out (PISO) shift register.



part (d):

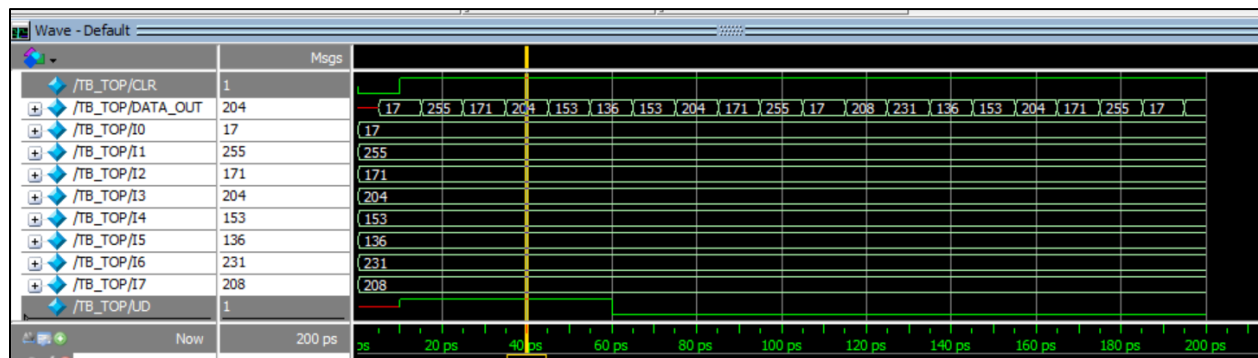
TOP

For this design, combine the 3-bit up/down counter (Module 1) with the 8-to-1 multiplexer (Module 2) to create a circuit such that the output of the counter controls the select lines of the multiplexer.

Code:

```
1  module TOP (CLOCK, CLR, UD, I0, I1, I2, I3, I4, I5, I6, I7, DATA_
2      //Write your code here
3
4      input CLOCK, CLR, UD;
5      input [7:0] I0, I1, I2, I3, I4, I5, I6, I7;
6      output [7:0] DATA_OUT;
7
8      wire [2:0] COUNT;
9
10     COUNTER c (CLOCK, CLR, UD, COUNT);
11     MUX81 m (COUNT, I0, I1, I2, I3, I4, I5, I6, I7, DATA_OUT);
12
13 endmodule
```

Output:



Analysis:

This code is error-free. It instantiates and interconnects both the MUX81 and COUNTER modules.

